TRADINGLAB 2.0 ? DOCUMENTO DE ARQUITECTURA

TradingLab 2.0 — Arquitectura propuesta

Arquitectura integral y procedimientos operativos del laboratorio

Versión: 2.2
Estado: Propuesta para validación de socios
Fecha: 7 de octubre de 2026
Documento precedente: `Trading_Lab_2_0_Arquitectura_Inicial.pdf`

1. Resumen ejecutivo

TradingLab 2.0 se organizará como un laboratorio de investigación de estrategias de trading reproducible, auditable y automatizable. La propuesta conserva las mejores decisiones de la arquitectura inicial —un repositorio Git, una base lógica central y múltiples proyectos— e incorpora fronteras más precisas para código, documentación, datos, experimentos y artefactos.

La arquitectura se apoya en cuatro sistemas con responsabilidades distintas:

1. **Git y GitHub:** código, configuraciones, documentación, migraciones y manifiestos pequeños.
2. **Catálogo transaccional:** proyectos, estrategias, experimentos, ejecuciones, métricas, decisiones y referencias a artefactos.
3. **Almacenamiento de artefactos:** datasets, reportes, exportaciones de MT5/SQX y otros archivos pesados o generados.
4. **Servicios y herramientas:** API, CLI, procesos de importación y adaptadores para agentes como Codex.

La unidad fundamental de reproducibilidad será el **run manifest**: un registro inmutable que identifica el código, datos, configuración, motor, condiciones de mercado y artefactos utilizados en cada ejecución. Para estrategias generadas en SQX se incorpora además un pipeline formal de preparación, compilación, validación, despliegue e incubación en MT5. La ejecución MT5 ocurre en una VPS externa sin acceso remoto desde TradingLab y se coordina mediante paquetes verificables intercambiados inicialmente por Google Drive.

2. Decisiones principales

2.1 Decisiones que se conservan de la versión inicial

- Un único repositorio Git para TradingLab mientras el sistema mantenga una única gobernanza.
- Una base lógica central para permitir análisis entre proyectos.
- Proyectos separados mediante identificadores y carpetas, no mediante repositorios o bases independientes.
- Git para cambios del laboratorio; la base para experimentos y resultados; la bitácora para contexto humano.
- PostgreSQL como objetivo razonable cuando haya concurrencia, automatizaciones y varios clientes.
- Codex como cliente del laboratorio, no como fundamento del sistema.
- `AGENTS.md` para reglas persistentes y `.agents/skills/` para procedimientos repetibles.

2.2 Cambios introducidos

Tema	Arquitectura inicial	Arquitectura propuesta
Directorios	Nombres con espacios y categorías generales	Rutas estables en `kebab-case` y unidades con responsabilidad definida
Datos	No se define una capa explícita	Datasets versionados y almacenamiento externo o local administrado
Artefactos	Ruta de archivo en la base	URI, hash, tamaño, tipo, backend y política de retención
Reproducibilidad	Commit, versión de motor, datos y parámetros	Manifiesto inmutable que incluye también estado Git, entorno, costes, zona horaria y semillas
Métricas	Pares `metric_name` / `metric_value`	Definiciones, tipos, unidades, scope y versión del cálculo

Tema	Arquitectura inicial	Arquitectura propuesta
Software	`Programming/` por categoría técnica	`apps/`, `packages/`, `db/`, `infra/` y `tools/`
Documentación	Documentación y sesiones globales y locales	Reglas explícitas de autoridad; ADR para decisiones y bitácora para historia
MCP	Acceso futuro al laboratorio	Adaptador fino sobre una API de dominio; no acceso SQL directo
Implementación	Diseñar toda la base primero	Validar un flujo vertical MT5/SQX de extremo a extremo
Bots SQX/MT5	Exportación y ejecución tratadas implícitamente	Pipeline trazable con `BotBuild`, gates de calidad, despliegue e incubación
VPS MT5	Acceso operativo implícito	Zona de confianza externa, sin acceso entrante, operada mediante runner local y paquetes verificables
Distribución	No definida	Google Drive inicialmente; releases inmutables y attestations de GitHub como evolución

2.3 Modificación 2.1 — Pipeline SQX a MT5

La versión 2.1 incorpora formalmente el proceso que transforma una estrategia candidata de SQX en un ejecutable identificable y aprobado para incubación en MT5. Se introduce `BotBuild` para representar una construcción concreta, `Environment` para describir el entorno de ejecución y `Deployment` para registrar qué build se instaló, dónde y con qué configuración.

El procedimiento completo se define en `sqx-mt5-incubation-pipeline.md`.

2.4 Modificación 2.2 — VPS MT5 aislada y TradingLab MT5 Runner

La versión 2.2 formaliza que TradingLab y MT5 se ejecutan en VPS diferentes y que la VPS TradingLab no tendrá acceso remoto, credenciales ni conectividad de administración hacia la VPS MT5. La segunda VPS podrá ejecutar un componente local llamado `TradingLab MT5 Runner`, instalado y operado por el socio.

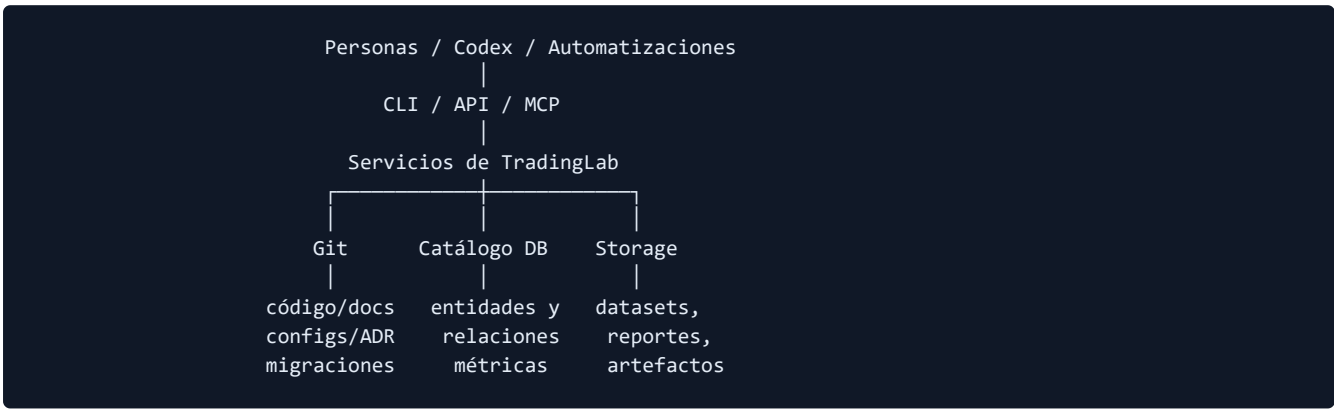
Se introducen `RunnerRelease`, `JobPackage`, `Acknowledgement` y `EvidenceBundle`. Google Drive se utiliza como transporte, nunca como fuente de verdad ni como autorización automática de ejecución. El workflow completo se define en `tradinglab-mt5-runner-workflow.md`.

3. Principios de diseño

1. **Una fuente de verdad por tipo de información.** No se duplicarán resultados, decisiones o configuraciones sin indicar cuál copia es autoritativa.
2. **Reproducibilidad antes que volumen.** Un resultado que no pueda vincularse a sus entradas exactas es observacional, no reproducible.
3. **Artefactos inmutables.** Un archivo registrado no se modifica; una corrección produce una nueva versión y un nuevo hash.
4. **Automatización idempotente.** Repetir una importación o registro no debe crear duplicados.
5. **Dominio independiente de herramientas.** MT5, SQX, PostgreSQL, GitHub o Codex son adaptadores del laboratorio, no su modelo conceptual.
6. **Seguridad por defecto.** Credenciales fuera de Git, acceso mínimo y operaciones sensibles auditables.
7. **Crecimiento incremental.** Primero un flujo real y verificable; después generalización, interfaz gráfica, MCP y automatización avanzada.

8. **Promoción mediante gates.** Un bot solo avanza de SQX a incubación cuando supera controles explícitos de preparación, compilación y validación.
9. **Aislamiento por zonas de confianza.** La automatización no requiere acceso remoto a MT5; cada zona conserva sus credenciales y responsabilidades.
10. **Pull verificado y aprobación local.** La VPS MT5 obtiene paquetes, verifica integridad y autoriza localmente su ejecución.

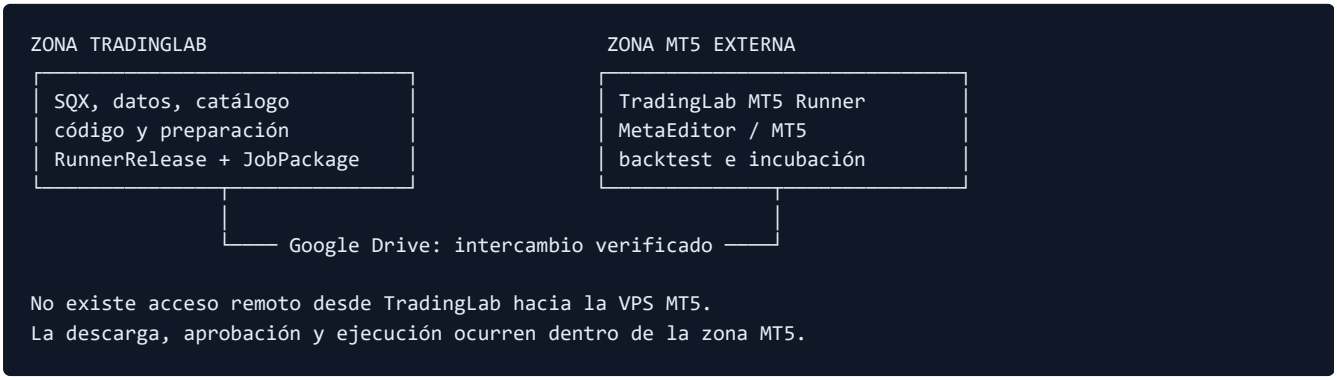
4. Vista lógica



El flujo de construcción y ejecución se integra de esta forma:



La frontera física y de seguridad queda definida así:



MCP será opcional y se implementará como adaptador de operaciones tipadas. Ningún agente debería escribir directamente en las tablas de producción ni manipular los archivos internos de PostgreSQL.

5. Fuentes de verdad

Información	Fuente autoritativa	Copias o vistas derivadas
Código y scripts	Git	Builds, paquetes y despliegues
Configuración declarativa	Git	Copia normalizada/hash en el catálogo
Esquema de base de datos	Migraciones en Git	Esquema desplegado
Estado de experimentos y runs	Catálogo	Dashboards y reportes
Métricas normalizadas	Catálogo	Agregaciones y visualizaciones
Dataset binario	Storage	Cachés locales
Identidad/versiones de dataset	Catálogo + manifiesto	Índices y vistas
Reportes MT5/SQX	Storage	Referencia y hash en catálogo
Exportación original de SQX	Storage	Referencia y hash en catálogo
Receta de preparación y componentes MQL5	Git	Código preparado generado
Estado y manifiesto de `BotBuild`	Catálogo	Informes y dashboards
Ejecutable `.ex5` y log de compilación	Storage	Referencia, hash y estado en catálogo
Configuración efectiva de incubación	Git + hash en catálogo	Copia desplegada en MT5
Estado de deployment e incubación	Catálogo	Bitácoras y dashboards
Código fuente del MT5 Runner	Git, rama principal	Paquete de release
`RunnerRelease` aprobado	Storage + manifiesto/hash	Copia de transporte en Google Drive
`JobPackage` emitido	Storage + catálogo	Copia de transporte en Google Drive
`Acknowledgement` y `EvidenceBundle`	Storage + catálogo	Copia recibida por Google Drive
Estado efectivo informado por VPS MT5	Catálogo con nivel de verificación	Dashboards y bitácora
Decisión arquitectónica	ADR en Git	Resumen enlazado desde sesiones
Historia de trabajo	Bitácora en Git	Resúmenes periódicos
Secretos	Gestor de secretos o entorno local	Nunca Git ni bitácora

6. Estructura propuesta del repositorio

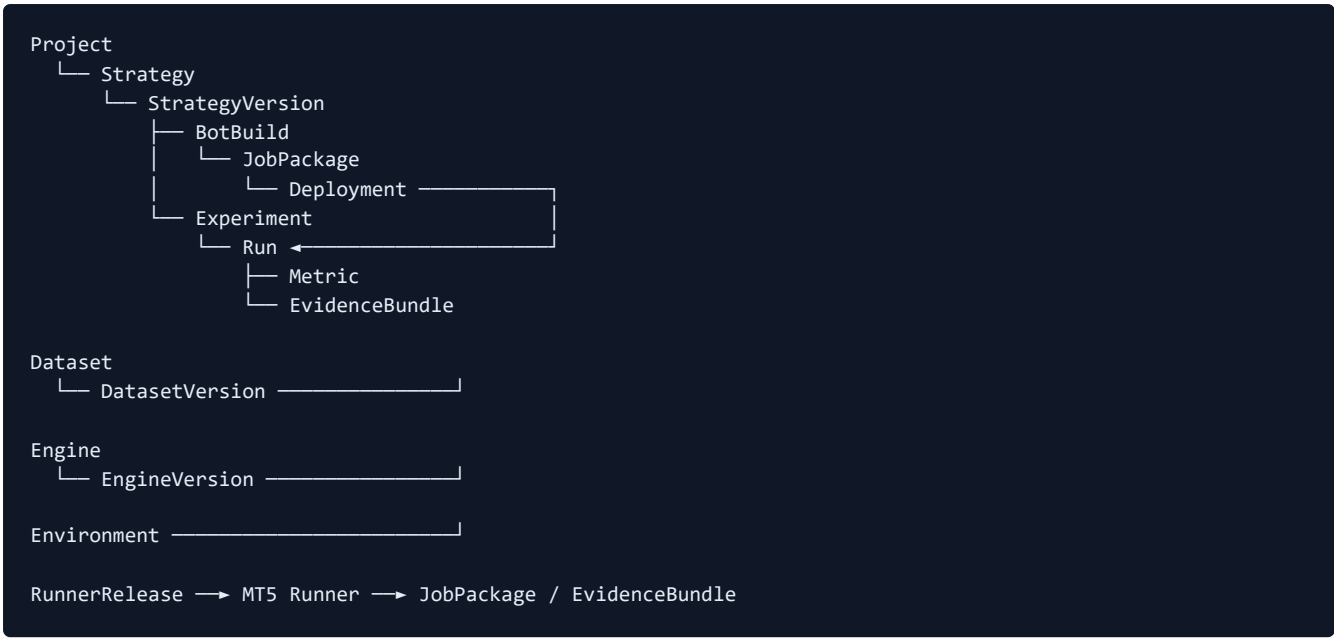
```
trading-lab/
├── README.md
├── AGENTS.md
├── .gitignore
├── .env.example
│
├── .agents/
│   └── skills/
│
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── knowledge/
│   └── sessions/
│
├── projects/
│   └── scalping-9-robots/
│       ├── README.md
│       ├── AGENTS.md              # solo si existen reglas específicas
│       ├── docs/
│       ├── configs/
│       │   ├── strategy/
│       │   ├── build/
│       │   └── incubation/
│       └── platforms/
│           ├── mt5/
│           │   ├── src/
│           │   ├── include/
│           │   ├── presets/
│           │   └── tests/
│           └── sqx/
│               ├── export-profiles/
│               ├── templates/
│               └── adapters/
│
├── apps/
│   ├── api/
│   ├── cli/
│   ├── worker/
│   └── mt5-runner/
│
├── packages/
│   ├── domain/
│   ├── ingestion/
│   ├── bot-pipeline/
│   ├── exchange-contracts/
│   └── reporting/
│
├── db/
│   ├── migrations/
│   ├── seeds/
│   └── README.md
│
├── infra/
└── tools/
```

Convenciones de rutas

- Usar minúsculas y `kebab-case` en carpetas y archivos operativos.
- No introducir nombres dependientes de una máquina o usuario.

- No guardar outputs masivos dentro del árbol versionado.
- Conservar el PDF inicial en la raíz durante la comparación; después podrá trasladarse a `docs/architecture/archive/` mediante una decisión explícita.
- El contenido fuente editable es canónico; un PDF generado es un entregable.

7. Modelo de dominio mínimo



Definiciones

- **Project:** objetivo de investigación con alcance y criterios propios.
- **Strategy:** identidad estable de una idea o familia de reglas.
- **StrategyVersion:** versión inmutable de código y configuración de una estrategia.
- **BotBuild:** construcción concreta y verificable que transforma una versión de estrategia en un ejecutable MT5.
- **Environment:** descripción versionada del terminal, broker, símbolo y condiciones donde se ejecutará el bot.
- **Deployment:** registro de qué `BotBuild` se instaló en qué `Environment`, con qué preset y durante qué periodo.
- **RunnerRelease:** paquete versionado del software que opera localmente en la VPS MT5.
- **JobPackage:** solicitud declarativa y verificable de compilación, backtest, deployment o incubación; nunca un comando arbitrario.
- **Acknowledgement:** confirmación emitida por la VPS MT5 al aceptar, rechazar o cambiar el estado de un job.
- **EvidenceBundle:** conjunto inmutable de manifiesto, logs, reportes y hashes devuelto por la VPS MT5.
- **Experiment:** hipótesis y protocolo que se desea evaluar.
- **Run:** ejecución concreta de un experimento bajo entradas exactas.
- **DatasetVersion:** snapshot identificable del mercado y sus transformaciones.
- **Metric:** observación normalizada asociada a un run y a un scope.
- **Artifact:** archivo inmutable generado o consumido por el laboratorio.

Entidades iniciales

El primer esquema debería limitarse a:

- `projects`

- `strategies`
- `strategy_versions`
- `bot_builds`
- `environments`
- `deployments`
- `runner_releases`
- `jobs`
- `job_acknowledgements`
- `evidence_bundles`
- `experiments`
- `runs`
- `datasets`
- `dataset_versions`
- `metric_definitions`
- `metrics`
- `artifacts`
- `run_artifacts`

Las tablas se ampliarán solamente después de validar un caso real. Se aplicarán claves foráneas, restricciones únicas, timestamps en UTC y migraciones versionadas.

8. Identificadores y estados

Las claves internas pueden ser UUID/ULID. Los nombres visibles no serán claves. Cada entidad tendrá además un `slug` estable cuando sea útil.

Estados iniciales sugeridos:

- Experimento: `draft`, `ready`, `running`, `completed`, `failed`, `cancelled`, `invalidated`.
- Run: `queued`, `running`, `succeeded`, `failed`, `cancelled`, `invalidated`.
- Estrategia: `research`, `candidate`, `validated`, `retired`.
- Bot build: `draft`, `prepared`, `compiling`, `compiled`, `failed`, `tested`, `approved`, `rejected`, `superseded`.
- Deployment: `planned`, `deployed`, `active`, `stopped`, `failed`, `rolled_back`.
- Job: `draft`, `published`, `received`, `approved`, `running`, `completed`, `failed`, `rejected`, `reconciled`.
- Evidence: `received`, `quarantined`, `verified`, `partially_verified`, `self_reported`, `rejected`, `imported`.

No se eliminarán resultados válidos para ocultar un fallo. Se marcarán como invalidados con motivo, autor y fecha.

9. Manifiesto reproducible de ejecución

Cada run producirá un manifiesto equivalente a:

```yaml
schema_version: 1
run_id: run_01...
experiment_id: exp_01...
strategy_version_id: stv_01...
bot_build_id: bld_01...
deployment_id: dep_01...
job_id: job_01...
evidence_bundle_id: evb_01...

source:
  git_commit: 8f31c2a...
  git_dirty: false

engine:
  name: mt5
  version: "5.00"
  build: "..."

environment:
  environment_id: env_01...
  broker_alias: "..."
  account_alias: "..."          # nunca credenciales
  terminal_instance: "..."

data:
  dataset_version_id: dsv_01...
  sha256: "..."
  timezone: UTC

execution:
  configuration_sha256: "..."
  executable_sha256: "..."
  preset_sha256: "..."
  execution_mode: external-runner
  random_seed: 12345
  started_at: "2026-10-07T18:00:00Z"
  completed_at: "2026-10-07T18:15:00Z"
  reported_at: "2026-10-07T18:20:00Z"
  received_at: "2026-10-07T18:25:00Z"

verification:
  status: verified
  operator_alias: "..."

market_assumptions:
  symbol: XAUUSD
  timeframe: M5
  broker: "..."
  spread_model: "..."
  commission_model: "..."
  slippage_model: "..."

artifacts:
  - role: report
    uri: "file:///..."          # podrá ser s3:// u otro backend
    sha256: "..."
    media_type: text/html
```

El sistema rechazará el cierre exitoso de un run si faltan campos obligatorios o si los hashes no coinciden.

10. Métricas

No se utilizará un único campo ambiguo `metric_value`. Cada métrica se asociará a una definición que indique:

- nombre canónico;
- descripción;
- tipo de dato;
- unidad;
- dirección deseable, cuando corresponda;
- versión de fórmula;
- motor u origen.

Cada valor incluirá:

- `run_id`;
- `metric_definition_id`;
- valor numérico, textual o booleano;
- scope: `in_sample`, `out_of_sample`, `full`, `monte_carlo`, etc.;
- dimensiones opcionales controladas;
- instante y procedencia de importación.

Las métricas crudas podrán conservarse como JSON en el artefacto de importación, pero las métricas utilizadas para filtrar o comparar deberán normalizarse.

11. Datos y artefactos

Desarrollo inicial

Se podrá usar un directorio local ignorado por Git, por ejemplo `_storage/`, siempre que:

- la base guarde una URI relativa o portable;
- exista un hash SHA-256;
- el directorio tenga respaldo;
- una verificación periódica detecte archivos faltantes o alterados.

Evolución

Cuando el volumen o el trabajo multiusuario lo exijan, los mismos contratos permitirán usar almacenamiento de objetos. La aplicación no deberá asumir que todo artefacto es un archivo local.

Google Drive como transporte

Google Drive se utilizará inicialmente para intercambiar `RunnerRelease`, `JobPackage`, `Acknowledgement` y `EvidenceBundle` entre zonas. Una vez recibido un paquete, TradingLab lo copiará a `inbox`, validará esquema, tamaño y hashes, y lo enviará a storage autoritativo o a `quarantine`. La ruta de Drive no será una referencia durable ni la prueba de que un job está autorizado.

La aparición de un archivo en Drive nunca provocará ejecución automática por sí sola. El MT5 Runner verificará el paquete y solicitará la aprobación local definida.

Registro mínimo de un artefacto

- identificador;
- URI;
- SHA-256;
- tamaño;
- media type;
- rol;
- backend;
- fecha de creación;
- política de retención;
- productor y run relacionado.

12. Procedimientos operativos

12.1 Crear un proyecto

1. Elegir un slug estable y comprobar que no exista.
2. Crear `projects/<slug>/README.md` con hipótesis, alcance y criterios de éxito.
3. Crear únicamente las carpetas necesarias.
4. Registrar el proyecto en el catálogo mediante CLI/API.
5. Añadir `AGENTS.md` local solo si hay reglas que realmente difieren de las globales.
6. Confirmar el cambio en Git y enlazar el commit al registro de creación.

12.2 Registrar una versión de estrategia

1. Verificar que el working tree esté limpio o declarar explícitamente el diff.
2. Normalizar la configuración.
3. Calcular el hash de configuración.
4. Registrar commit, configuración y relación con la versión precedente.
5. Marcar la versión como inmutable una vez utilizada por un run.

12.3 Ejecutar un experimento

1. Formular hipótesis y criterio de aceptación.
2. Asociar versión de estrategia, dataset, motor y configuración.
3. Validar prerrequisitos y crear el run en estado `queued`.
4. Ejecutar y conservar logs.
5. Calcular hashes y registrar artefactos.
6. Importar métricas mediante un proceso idempotente.
7. Validar consistencia y cerrar el run.
8. Documentar la interpretación humana sin modificar el resultado bruto.

12.4 Importar resultados de MT5 o SQX

1. Copiar el output a un área de ingreso temporal.
2. Detectar formato y versión del motor.
3. Validar campos obligatorios.

4. Calcular hash antes de transformar.
5. Guardar el original como artefacto inmutable.
6. Parsear hacia un modelo intermedio versionado.
7. Normalizar métricas y unidades.
8. Ejecutar controles de totales y rangos.
9. Registrar todo en una transacción o revertirlo.
10. Mover el artefacto a su ubicación definitiva y emitir un resumen.

12.5 Preparar y construir un bot SQX para MT5

1. Registrar la estrategia candidata y congelar su `StrategyVersion`.
2. Exportar desde SQX y guardar el archivo original como artefacto inmutable.
3. Aplicar una receta de preparación versionada; cualquier cambio manual debe quedar en Git.
4. Generar el código MQL5 preparado y calcular su hash.
5. Compilar con una versión registrada de MetaEditor y conservar el log completo.
6. Registrar un `BotBuild` con hashes de entrada, código preparado, preset y `.ex5`.
7. Rechazar builds con errores o que incumplan la política de warnings.

12.6 Validar y aprobar un BotBuild

1. Verificar dependencias, Magic Number, símbolo, timeframe y límites de riesgo.
2. Ejecutar inicialización y smoke backtest en MT5.
3. Comprobar apertura/cierre de órdenes, logs y manejo de errores.
4. Comparar el comportamiento esencial con la estrategia de origen en SQX.
5. Registrar evidencias y aprobar o rechazar el build. Un build aprobado es inmutable.

12.7 Desplegar e incubar en MT5

1. Seleccionar un `Environment` externo autorizado y verificar su configuración declarada.
2. Crear un `JobPackage` que vincule build, preset, entorno y acciones permitidas.
3. Publicarlo en el canal de Google Drive con manifiesto y checksums.
4. El operador de la VPS MT5 lo descarga, verifica y aprueba localmente.
5. El MT5 Runner instala o compila, crea el `Deployment` informado e inicia la incubación.
6. El runner captura logs, métricas, incidentes y cambios de estado.
7. Genera un `EvidenceBundle` y lo devuelve por el canal acordado.
8. TradingLab verifica, importa y reconcilia la evidencia antes de cerrar el run.
9. El operador retira o revierte el deployment cuando corresponda.

El contrato del pipeline se encuentra en `sqx-mt5-incubation-pipeline.md`; la operación del runner y Google Drive se define en `tradinglab-mt5-runner-workflow.md`.

12.8 Publicar una versión del MT5 Runner

1. Construir desde `main` y un commit limpio; no utilizar una rama de servidor permanente.
2. Ejecutar tests y generar un paquete autocontenido.
3. Crear `release-manifest`, versión y checksums.
4. Revisar y aprobar el paquete.
5. Guardarlo en storage y publicar una copia en `tradinglab-to-mt5/runner-releases/`.
6. Conservar la versión anterior para rollback.

12.9 Procesar un JobPackage en la VPS MT5

1. Copiar el job desde Drive a un inbox local.
2. Validar esquema, versión mínima del runner, tamaños y hashes.
3. Rechazar acciones o rutas fuera de la allowlist.
4. Mostrar el plan y obtener la aprobación local requerida.
5. Ejecutar con usuario de privilegio mínimo y registrar cada etapa.
6. Emitir acknowledgements y un evidence bundle; nunca escribir directamente en el catálogo TradingLab.

12.10 Ingerir un EvidenceBundle

1. Copiar el paquete recibido a `inbox`; no procesarlo directamente desde Drive.
2. Tratarlo como entrada externa y validar formato, tamaño, rutas y hashes.
3. Enviar paquetes inválidos a `quarantine` sin ejecutarlos ni abrir contenido activo.
4. Conservar el original en storage.
5. Reconciliarlo con `job_id`, build, preset y entorno esperados.
6. Asignar nivel `verified`, `partially_verified`, `self_reported` o `rejected`.
7. Importar métricas idempotentemente y cerrar el job/run cuando corresponda.

12.11 Cerrar una sesión de trabajo

1. Registrar objetivo, trabajo realizado y evidencias.
2. Enlazar proyectos, experimentos, runs, ADR y commits relevantes.
3. Anotar decisiones y preguntas abiertas.
4. Separar claramente hechos, inferencias y próximos pasos.
5. No incluir secretos ni copiar resultados que ya sean autoritativos en la base.
6. Guardar la entrada en `docs/sessions/YYYY-MM-DD-<tema>.md`.

12.12 Adoptar una decisión arquitectónica

1. Crear un ADR con contexto, alternativas y consecuencias.
2. Asignar estado `proposed`, `accepted`, `superseded` o `rejected`.
3. Obtener la aprobación requerida.
4. Enlazar la decisión desde arquitectura y bitácora.
5. No reescribir un ADR aceptado; reemplazarlo mediante otro ADR.

12.13 Cambiar la base de datos

1. Crear una migración reversible cuando sea posible.
2. Probarla sobre una copia o entorno aislado.
3. Ejecutar validaciones de integridad.
4. Documentar cambios destructivos y respaldo requerido.
5. Aplicar mediante una identidad controlada, nunca editando archivos internos de PostgreSQL.

12.14 Copias de seguridad y recuperación

1. Respaldar catálogo y storage como conjuntos coordinados.
2. Registrar fecha, versión del esquema y checksums.
3. Cifrar copias que contengan información sensible.
4. Probar restauraciones periódicamente.

5. Definir RPO y RTO antes de utilizar el laboratorio para operaciones críticas.

13. Git y GitHub

Contenido versionado

- código, tests y scripts;
- documentación fuente y ADR;
- bitácora;
- configuraciones sin secretos;
- migraciones y seeds no sensibles;
- manifiestos pequeños;
- definición de skills y reglas de agentes.

Contenido excluido

- `.env` y credenciales;
- ejecutables y builds generados;
- bases físicas;
- cachés y logs temporales;
- `_storage/` y datasets pesados;
- informes generados, salvo excepciones documentadas;
- archivos específicos del IDE o sistema operativo.

Estrategia inicial

- Rama principal protegida: `main`.
- Cambios mediante ramas cortas y pull requests cuando participe más de una persona.
- Commits pequeños con intención clara.
- Tags para hitos de arquitectura o releases.
- Revisión obligatoria para migraciones destructivas, cambios de seguridad y contratos de datos.
- El MT5 Runner se desarrolla en `main` y se distribuye como paquete versionado; no tendrá una rama permanente por servidor.
- Durante la fase inicial, la VPS MT5 no tendrá credenciales del repositorio principal.

La inicialización y conexión con GitHub se realizarán después de validar este documento. Antes de ejecutar `git init` se confirmarán propietario, nombre del repositorio, visibilidad, URL remota y estrategia de rama.

14. Seguridad y gobierno

- Utilizar `.env.example` con nombres de variables, nunca valores reales.
- Mantener credenciales en variables de entorno o gestor de secretos.
- Aplicar mínimo privilegio a API, DB y storage.
- Separar lectura, ingestión, administración y migraciones.
- Auditar cambios de estado e invalidaciones.
- Evaluar licencias y restricciones de redistribución de datos de mercado.
- Sanitizar entradas provenientes de informes o fuentes externas antes de exponerlas a agentes.

- Requerir aprobación humana para borrados, despliegues y operaciones irreversibles.
- Mantener cerrada la conectividad de administración desde TradingLab hacia la VPS MT5.
- Ejecutar el runner con usuario sin privilegios administrativos y acceso limitado a sus carpetas y a MT5.
- No aceptar comandos PowerShell arbitrarios dentro de un job; solo acciones declarativas permitidas por esquema.
- No ejecutar automáticamente paquetes por el solo hecho de aparecer en Google Drive.
- Verificar hashes, tamaño, rutas y versión antes de instalar o ejecutar.
- No guardar credenciales de GitHub, Drive, MT5 o broker dentro de paquetes, manifests o evidencias.

15. Codex, AGENTS y skills

El `AGENTS.md` raíz será breve y contendrá:

- mapa del repositorio;
- comandos de build y test;
- reglas de seguridad;
- fuentes de verdad;
- criterios de finalización;
- disparadores de skills obligatorias.

Las skills se usarán para procedimientos repetibles como importar informes, validar una estrategia, registrar un experimento, ejecutar migraciones y cerrar sesiones. Las acciones deterministas irán en scripts; la interpretación y comparación permanecerán bajo revisión del modelo y de las personas.

Los archivos `AGENTS.md` de proyecto solo introducirán diferencias locales. No duplicarán las reglas globales.

16. Observabilidad y calidad

Cada servicio deberá emitir logs estructurados con `correlation_id`, `project_id`, `experiment_id` y `run_id` cuando correspondan. Se medirán:

- ejecuciones correctas/fallidas;
- duración de importaciones;
- artefactos faltantes o corruptos;
- duplicados rechazados;
- versión de parsers;
- migraciones aplicadas.

La calidad mínima incluirá tests unitarios del dominio, fixtures representativas de MT5/SQX, tests de migraciones y al menos un test de extremo a extremo del flujo piloto.

17. Plan de implementación

Fase 0 — Aprobación

- Comparar las dos arquitecturas.
- Resolver las decisiones abiertas.
- Aceptar la versión mediante ADR.

- Inicializar Git y conectar GitHub.

Fase 1 — Fundación

- Crear estructura mínima, README, AGENTS y `.gitignore`.
- Definir glosario, identificadores y run manifest.
- Definir build manifest, política de warnings y gates del pipeline SQX/MT5.
- Definir esquemas de `RunnerRelease`, `JobPackage`, `Acknowledgement` y `EvidenceBundle`.
- Definir permisos y estructura de intercambio en Google Drive.
- Elegir runtime y herramientas de migración.
- Crear esquema mínimo del catálogo.

Fase 2 — Flujo piloto SQX a MT5

- Seleccionar un proyecto y una estrategia candidata real de SQX.
- Registrar proyecto, estrategia, versión y exportación original.
- Preparar, compilar y validar un `BotBuild` reproducible.
- Empaquetar manualmente la primera versión del MT5 Runner y entregarla por Google Drive.
- Crear el primer JobPackage, procesarlo con aprobación local en la VPS MT5 e iniciar una incubación.
- Ingerir resultados, conservar artefactos y normalizar métricas.
- Demostrar trazabilidad desde la métrica hasta evidence bundle, job, deployment, `.ex5`, build, código preparado, exportación SQX, configuración y datos.

Fase 3 — Operación repetible

- Incorporar CLI/API estable.
- Automatizar generación y validación de paquetes sin automatizar todavía la aprobación local.
- Implementar el MT5 Runner como herramienta local actualizable y reversible.
- Automatizar validaciones, backups y controles de integridad.
- Crear las primeras skills basadas en procedimientos ya probados.

Fase 4 — Escala

- PostgreSQL multiusuario si no se adoptó antes.
- Storage de objetos.
- Workers, colas y ejecución concurrente.
- MCP, dashboards y análisis transversales.
- Releases inmutables y attestations de GitHub para el runner, si resultan compatibles con la política de seguridad.

18. Criterios de aceptación del piloto

La arquitectura se considerará validada cuando un caso real permita:

1. Identificar inequívocamente proyecto, estrategia, versión, experimento y run.
2. Recuperar el reporte original y verificar su hash.
3. Consultar métricas normalizadas con unidades y scope.
4. Conocer commit, configuración, motor y dataset utilizados.
5. Repetir la importación sin duplicar registros.
6. Detectar un artefacto modificado o faltante.

7. Restaurar catálogo y artefactos desde una copia de seguridad.
8. Reconstruir el contexto humano desde ADR y bitácora.
9. Identificar exactamente el `BotBuild` y `.ex5` utilizados en MT5 mediante hashes.
10. Reproducir la preparación y compilación o explicar cualquier paso manual registrado.
11. Demostrar que el build superó los gates antes de la incubación.
12. Relacionar el deployment y el entorno MT5 con el run de incubación.
13. Instalar y revertir una versión identificable del MT5 Runner sin acceso remoto desde TradingLab.
14. Procesar un job declarativo sin aceptar comandos arbitrarios.
15. Verificar e ingerir el evidence bundle retornado mediante Google Drive.
16. Demostrar que Drive puede dejar de estar disponible sin perder la evidencia autoritativa.

19. Decisiones abiertas para los socios

1. Nombre y visibilidad del repositorio GitHub.
2. Organización o cuenta propietaria del repositorio.
3. PostgreSQL desde el inicio o catálogo local durante el piloto.
4. Ubicación y respaldo del storage inicial.
5. Primer proyecto y primer reporte para el flujo piloto.
6. Retención de informes y datasets originales.
7. Política de ramas y revisiones.
8. Tecnología principal para API, CLI y workers.
9. Nivel de acceso que tendrán agentes y automatizaciones.
10. Datos o configuraciones sujetos a restricciones contractuales.
11. Versión/build de MetaEditor admitida y estrategia para fijarla.
12. Política de warnings de compilación y criterios de equivalencia SQX/MT5.
13. Primer entorno MT5 de incubación y responsables de aprobación/despliegue.
14. Límites de riesgo y requisitos técnicos obligatorios antes de incubar.
15. Permisos y propietario de las carpetas de intercambio de Google Drive.
16. Usuario, directorios y privilegios efectivos del MT5 Runner.
17. Acciones que requerirán aprobación humana en la VPS MT5.
18. Frecuencia de comprobación de jobs y política de expiración.
19. Momento y requisitos para migrar a releases inmutables de GitHub.

20. Conclusión

TradingLab 2.0 debe construirse como un sistema de evidencia experimental, no como una colección de carpetas o reportes. El monorepo y el catálogo central ofrecen una base adecuada, siempre que el almacenamiento, las fuentes de verdad y la reproducibilidad tengan contratos explícitos.

La siguiente acción, una vez aprobada esta propuesta, será inicializar el repositorio Git, conectarlo con GitHub y ejecutar la Fase 1 sin anticipar componentes que todavía no hayan sido validados por el flujo piloto.